

系统开发工具基础

第三周实验报告

作者：魏尚儒

学号：25020007136

链接：

<https://github.com/1583e4d-Rn/2026summer-reports>

2026 年 9 月 11 日

目录

1	实验内容	2
1.1	从源码构建并在干净环境安装 Wheel	2
1.2	让编程智能体进入可验证的修复循环	5
1.3	把“无法处理”的协作材料改成可执行信息	9
1.4	修复一个可复现的线性回归训练循环	10
2	课后练习	12
2.1	Packaging and Shipping Code	12
2.1.1	用 pip 安装与卸载第三方包	12
2.1.2	用 python -m build 构建 wheel 与 sdist	13
2.1.3	从本地 wheel 安装包	13
2.2	智能体编程	13
2.2.1	测试驱动的智能体修复流程	13
2.2.2	用 git diff 审查智能体的改动	13
2.3	不止于代码	14
2.3.1	编写规范的 Git 提交信息	14
2.3.2	编写可复现的 Issue 报告	14
2.4	Python 与 PyTorch	14
2.4.1	PyTorch 张量的创建与运算	14
2.4.2	自动求导与梯度清零	15
2.4.3	用 nn.Linear 搭建最小训练循环	15
3	感悟	16

1 实验内容

1.1 从源码构建并在干净环境安装 Wheel

本题要求从源码构建一个最小 Python 命令行包，并验证 wheel 的安装，详细如下：

- 创建 src/greetlab/__init__.py、给定的 cli.py 和 pyproject.toml，并把“学号”替换为自己的学号。
- 在已安装 build 工具的课程环境中运行 python -m build，生成 wheel。
- 新建一个干净虚拟环境，只从生成的 wheel 安装。
- 切换到 q09 之外运行 sdt-greet -name < 学号 >，确认输出正确。

给定内容：

```
# src/greetlab/cli.py
import argparse
def main():
    p = argparse.ArgumentParser()
    p.add_argument("--name", required=True)
    a = p.parse_args()
    print(f"Hello, {a.name}!")
```

```
# pyproject.toml
[build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"
[project]
name = "greetlab-25020007136"
```

```
version = "0.1.0"
[project.scripts]
sdt-greet = "greetlab.cli:main"
```

创建项目结构

将“学号”替换为 25020007136，并创建包目录与空的 `__init__.py`：

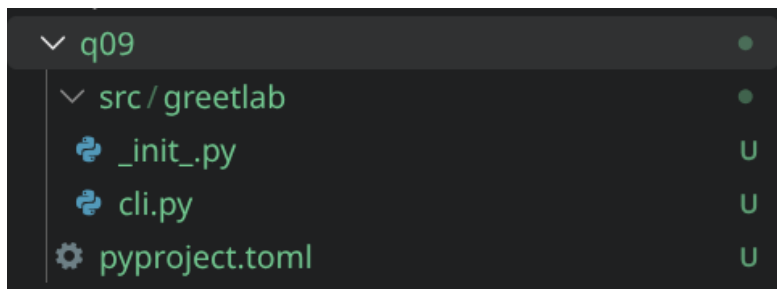
```
mkdir -p q09/src/greetlab
touch q09/src/greetlab/__init__.py
```

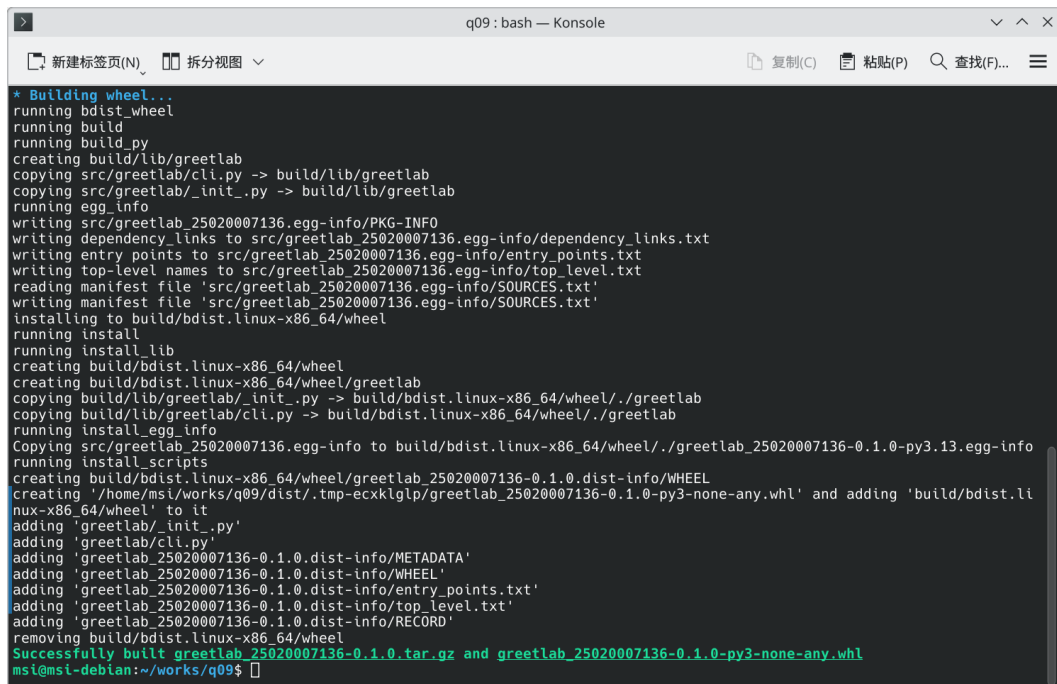
构建 wheel

在 q09 目录下运行 `python -m build`，生成 wheel：

```
cd q09
python3 -m build
```

构建完成后生成 dist 下的 wheel，效果如下：





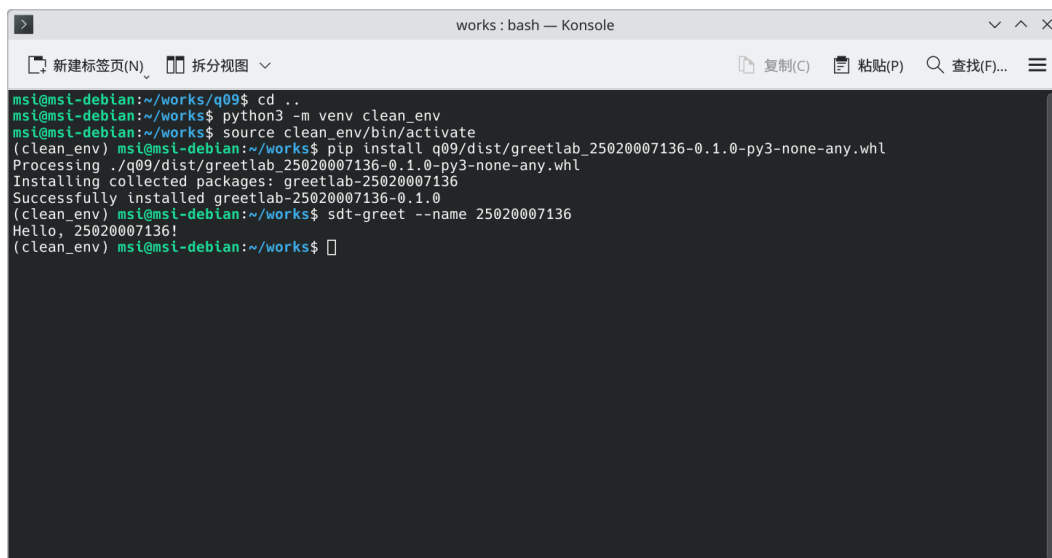
```
q09: bash — Konsole
新建标签页(N) 拆分视图
* Building wheel...
running bdist_wheel
running build
running build_py
creating build/lib/greetlab
copying src/greetlab/cli.py -> build/lib/greetlab
copying src/greetlab/_init_.py -> build/lib/greetlab
running egg_info
writing src/greetlab_25020007136.egg-info/PKG-INFO
writing dependency_links to src/greetlab_25020007136.egg-info/dependency_links.txt
writing entry points to src/greetlab_25020007136.egg-info/entry_points.txt
writing top-level names to src/greetlab_25020007136.egg-info/top_level.txt
reading manifest file 'src/greetlab_25020007136.egg-info/SOURCES.txt'
writing manifest file 'src/greetlab_25020007136.egg-info/SOURCES.txt'
installing to build/bdist.linux-x86_64/wheel
running install
running install_lib
creating build/bdist.linux-x86_64/wheel
creating build/bdist.linux-x86_64/wheel/greetlab
copying build/lib/greetlab/_init_.py -> build/bdist.linux-x86_64/wheel/./greetlab
copying build/lib/greetlab/cli.py -> build/bdist.linux-x86_64/wheel/./greetlab
running install_egg_info
Copying src/greetlab_25020007136.egg-info to build/bdist.linux-x86_64/wheel/./greetlab_25020007136-0.1.0-py3.13.egg-info
running install_scripts
creating build/bdist.linux-x86_64/wheel/greetlab_25020007136-0.1.0.dist-info/WHEEL
creating '/home/msi/works/q09/dist/.tmp-ecxklglp/greetlab_25020007136-0.1.0-py3-none-any.whl' and adding 'build/bdist.linux-x86_64/wheel' to it
adding 'greetlab/_init_.py'
adding 'greetlab/cli.py'
adding 'greetlab_25020007136-0.1.0.dist-info/METADATA'
adding 'greetlab_25020007136-0.1.0.dist-info/WHEEL'
adding 'greetlab_25020007136-0.1.0.dist-info/entry_points.txt'
adding 'greetlab_25020007136-0.1.0.dist-info/top_level.txt'
adding 'greetlab_25020007136-0.1.0.dist-info/RECORD'
removing build/bdist.linux-x86_64/wheel
Successfully built greetlab_25020007136-0.1.0.tar.gz and greetlab_25020007136-0.1.0-py3-none-any.whl
msi@msi-debian:~/works/q09$
```

在干净环境安装并运行

新建一个干净虚拟环境，只从生成的 wheel 安装，并切换到 q09 之外运行：

```
python3 -m venv /tmp/greetlab-venv
/tmp/greetlab-venv/bin/pip install dist/greetlab_25020007136-0.1.0-py3-none-any.whl
cd /tmp
/tmp/greetlab-venv/bin/sdt-greet --name 25020007136
```

输出 Hello, 25020007136!，效果如下：



```
msi@msi-debian:~/works/q09$ cd ..
msi@msi-debian:~/works$ python3 -m venv clean_env
msi@msi-debian:~/works$ source clean_env/bin/activate
(clean_env) msi@msi-debian:~/works$ pip install q09/dist/greetlab_25020007136-0.1.0-py3-none-any.whl
Processing ./q09/dist/greetlab_25020007136-0.1.0-py3-none-any.whl
Installing collected packages: greetlab-25020007136
Successfully installed greetlab-25020007136-0.1.0
(clean_env) msi@msi-debian:~/works$ sdt-greet --name 25020007136
Hello, 25020007136!
(clean_env) msi@msi-debian:~/works$
```

1.2 让编程智能体进入可验证的修复循环

本题要求复用第 9 题的 greetlab 包，使用编程智能体完成一个小型测试驱动修复，具体如下：

- 复制 q09 为 q10，添加一个失败测试：当 name 只含空白字符时，main 应以 SystemExit(2) 结束。
- 先运行测试确认失败，再向编程智能体说明目标、约束和测试命令，要求它修改实现并运行测试。
- 人工检查 diff，撤销无关修改，并再次运行测试。
- 在 ai_log.md 中用不超过 5 行记录核心提示、智能体改动和人工验证。

编写失败测试

复制 q09 为 q10，并添加空白姓名的失败测试 test_cli.py：

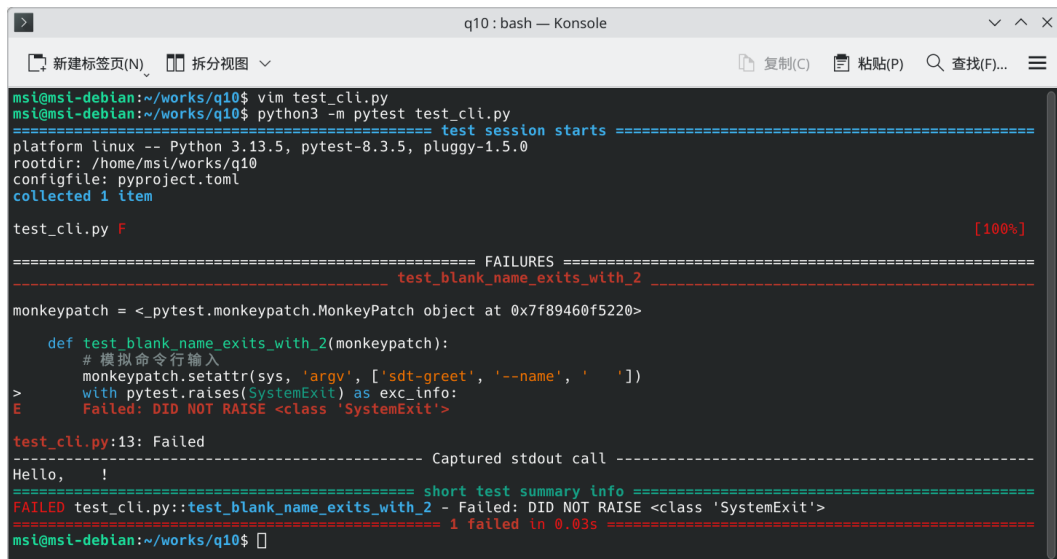
```
import sys
import os

# 将 q10/src 目录加入到 Python 搜索路径中
sys.path.insert(0, os.path.abspath(os.path.join(os.path.dirname(__file__), 'src')))

import pytest
from greetlab.cli import main

def test_blank_name_exits_with_2(monkeypatch):
    # 模拟命令行输入
    monkeypatch.setattr(sys, 'argv', ['sdt-greet', '--name', ''])
    with pytest.raises(SystemExit) as exc_info:
        main()
    # 验证退出码为 2
    assert exc_info.value.code == 2
```

先运行测试确认失败，效果如下：



```
msi@msi-debian:~/works/q10$ vim test_cli.py
msi@msi-debian:~/works/q10$ python3 -m pytest test_cli.py
===== test session starts =====
platform linux -- Python 3.13.5, pytest-8.3.5, pluggy-1.5.0
rootdir: /home/msi/works/q10
configfile: pyproject.toml
collected 1 item

test_cli.py F [100%]

===== FAILURES =====
test_cli.py::test_blank_name_exits_with_2
monkeypatch = <_pytest.monkeypatch.MonkeyPatch object at 0x7f89460f5220>

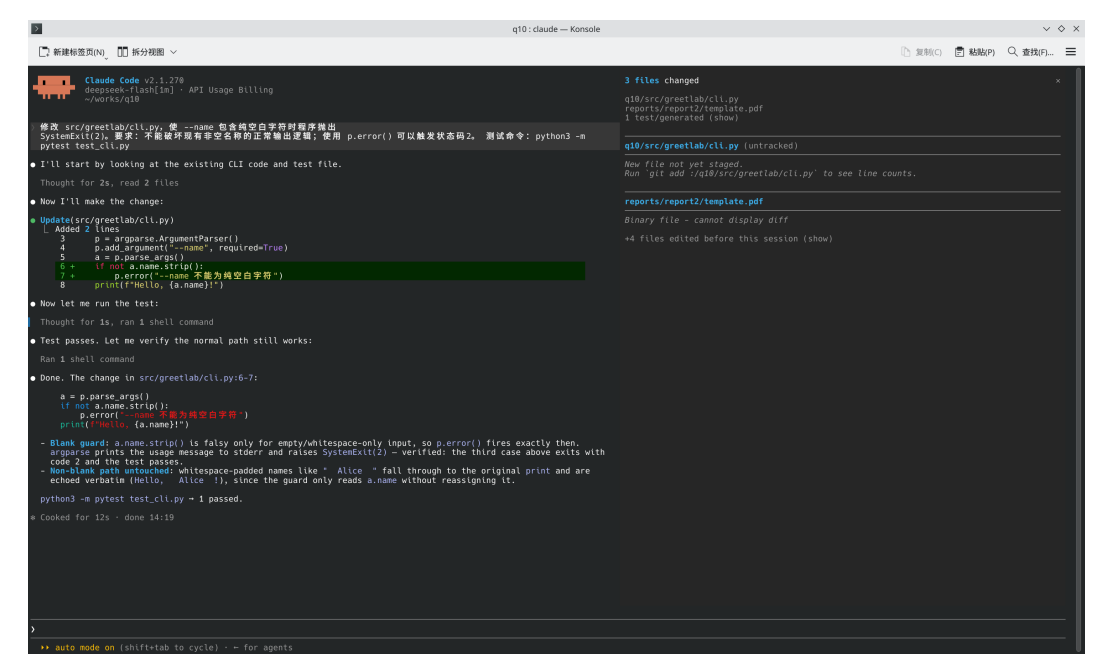
def test_blank_name_exits_with_2(monkeypatch):
    # 模拟命令行输入
    monkeypatch.setattr(sys, 'argv', ['sdt-greet', '--name', ''])
    with pytest.raises(SystemExit) as exc_info:
        >
E       Failed: DID NOT RAISE <class 'SystemExit'>

test_cli.py:13: Failed
----- Captured stdout call -----
Hello, !
===== short test summary info =====
FAILED test_cli.py::test_blank_name_exits_with_2 - Failed: DID NOT RAISE <class 'SystemExit'>
===== 1 failed in 0.03s =====
msi@msi-debian:~/works/q10$
```

让智能体修复

向编程智能体说明目标（纯空白 name 应抛出 SystemExit(2)）、约束（不得破坏正常问候）和测试命令（pytest），要求其修改实现。智能体在 main 中补充了空白校验逻辑：

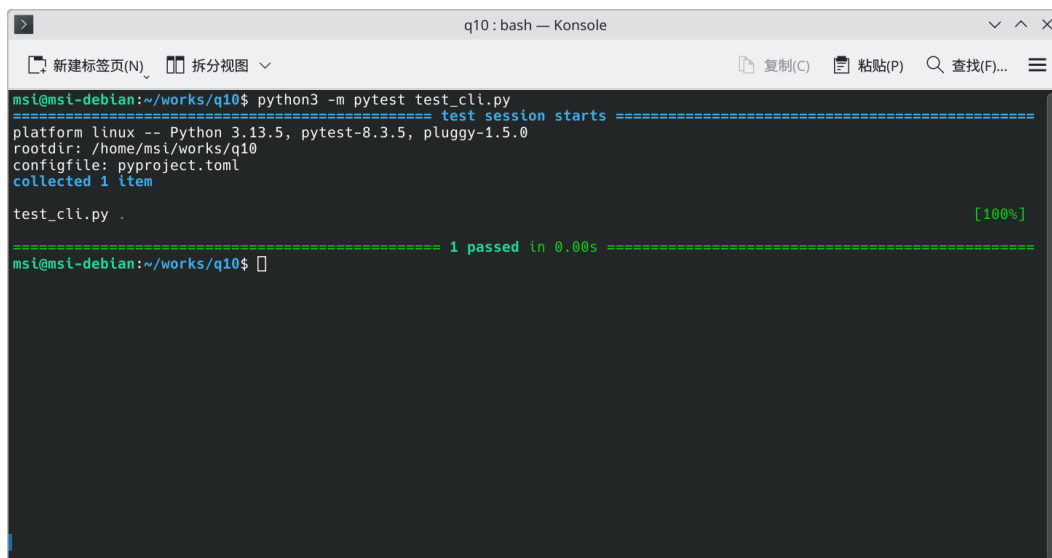
```
import argparse
def main():
    p = argparse.ArgumentParser()
    p.add_argument("--name", required=True)
    a = p.parse_args()
    if not a.name.strip():
        p.error("--name 不能为纯空白字符")
    print(f"Hello, {a.name}!")
```



人工审查与验证

审查 diff，确认无副作用修改后再次运行测试，测试通过；并用 ai_log.md 简要记录：

- 核心提示：让纯空白 name 抛出 SystemExit(2)。
 - 智能体改动：在 main 函数中增加 if not a.name.strip(): p.error(...)。
 - 人工验证：审查 diff 无副作用，pytest 测试通过 (Exit code 2)。



```
q10: bash — Konsole
新建标签页(N)  拆分视图
msi@msi-debian:~/works/q10$ python3 -m pytest test_cli.py
===== test session starts =====
platform linux -- Python 3.13.5, pytest-8.3.5, pluggy-1.5.0
rootdir: /home/msi/works/q10
configfile: pyproject.toml
collected 1 item

test_cli.py . [100%]

===== 1 passed in 0.00s =====
msi@msi-debian:~/works/q10$
```

1.3 把“无法处理”的协作材料改成可执行信息

本题要求围绕“空姓名仍输出问候语”这一缺陷，改写三段质量较差的协作材料，详细如下：

- 在 communication.md 中重写 Issue，包含环境、复现命令、期望结果和实际结果；未知信息写“待确认”。
- 重写提交信息：标题使用祈使语气，正文说明问题和解决方案。
- 重写评审意见：指出具体行为、风险和建议动作，并标注 Blocking、Suggestion 或 Nit。
- 全文控制在 400 字以内，保持专业、简洁。

改写后的 communication.md 如下：

```
### Issue: 空姓名仍输出问候语
- ** 环境 **: Windows 10, Python 3.10+
```

```
- ** 复现命令 **: sdt-greet --name "  "
```

```
- ** 预期结果 **: 报错并以非零状态码 (如 2) 退出。
```

```
- ** 实际结果 **: 输出 "Hello,    !", 并以 0 退出。
```

```
- ** 未知信息 **: 是否需考虑除空格外的其他不可见字符?(待确认)
```

提交信息

fix: 拦截纯空白姓名的无效输入

当前 `--name` 未校验空白字符, 导致输出错误问候语且退出码为 0。
通过剥离字符串并校验空值, 若为空则调用 `p.error()` 强制退出并返回状态码 2, 确保 CLI 交互严谨性。

评审意见

```
- ** 行为 **: if not a.name: 无法拦截包含空格或制表符的输入。
```

```
- ** 风险 **: 自动化调用方无法捕获错误, 用户体验差。(Blocking)
```

```
- ** 建议动作 **: 修改为 if not a.name.strip(): p.error("Name cannot be blank")。
```

1.4 修复一个可复现的线性回归训练循环

本题要求补全并完成一次 PyTorch CPU 训练, 详细如下:

- 补全训练循环, 正确调用 `zero_grad`、`backward` 和 `step`。
- 训练后调用 `model.eval()`, 并在 `torch.no_grad()` 中计算最终损失。
- 打印最终损失、`weight` 和 `bias`; 最终损失应小于 0.001。
- 不得直接把 `weight` 和 `bias` 赋值为 3 和 -1。

补全后的 `train.py` 如下:

```
import torch
```

```
from torch import nn

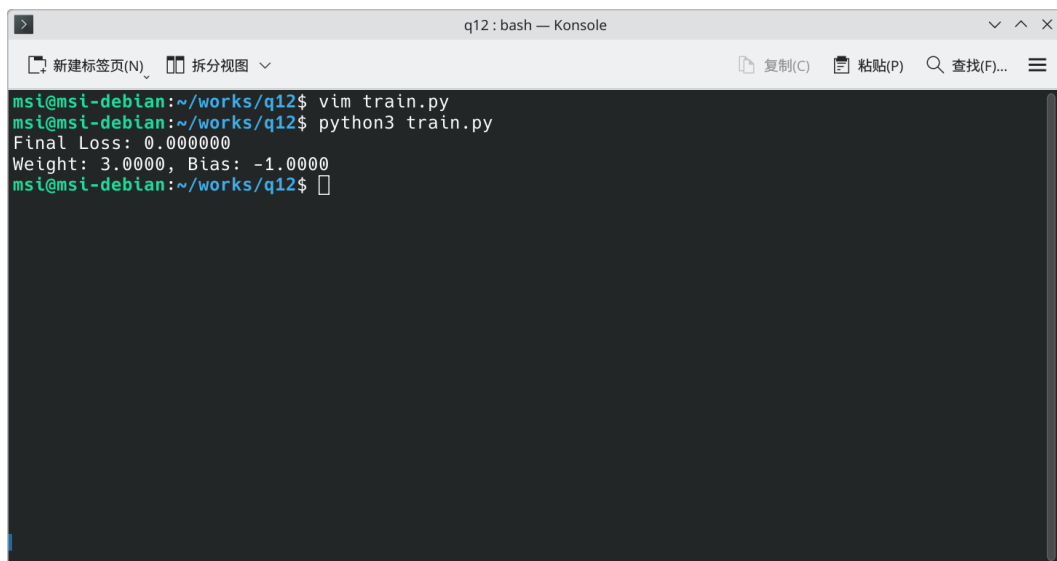
torch.manual_seed(20260907)
x = torch.linspace(-1, 1, 101).reshape(-1, 1)
y = 3 * x - 1
model = nn.Linear(1, 1)
loss_fn = nn.MSELoss()
opt = torch.optim.SGD(model.parameters(), lr=0.1)

for _ in range(200):
    pred = model(x)
    loss = loss_fn(pred, y)
    opt.zero_grad()
    loss.backward()
    opt.step()

model.eval()
with torch.no_grad():
    final_pred = model(x)
    final_loss = loss_fn(final_pred, y)
    weight = model.weight.item()
    bias = model.bias.item()
    print(f"Final Loss: {final_loss.item():.6f}")
    print(f"Weight: {weight:.4f}, Bias: {bias:.4f}")

assert final_loss.item() < 0.001, "Loss is too high!"
```

训练最终损失小于 0.001, weight 与 bias 分别收敛到约 3 和 -1。运行效果如下:



```
q12: bash — Konsole
新建标签页(N) 拆分视图
msi@msi-debian:~/works/q12$ vim train.py
msi@msi-debian:~/works/q12$ python3 train.py
Final Loss: 0.000000
Weight: 3.0000, Bias: -1.0000
msi@msi-debian:~/works/q12$
```

2 课后练习

2.1 Packaging and Shipping Code

2.1.1 用 pip 安装与卸载第三方包

pip 是 Python 的包管理工具，可在线安装、卸载并查看已安装的包，相关内容可参考课程资源中的 runoob Python 教程。

```
pip install requests          # 安装包
pip install requests==2.31.0 # 安装指定版本
pip uninstall requests -y     # 卸载包
pip list                      # 查看已安装的包
```

2.1.2 用 python -m build 构建 wheel 与 sdist

python -m build 会依据 pyproject.toml 同时生成 wheel 和源码包 sdist，是发布 Python 包的标准方式。

```
python3 -m build
# 生成 dist/*.whl 与 dist/*.tar.gz
```

2.1.3 从本地 wheel 安装包

在离线或已构建场景下，可直接用 pip 从本地.whl 文件安装，而不依赖网络源。

```
pip install dist/greetlab-25020007136-0.1.0-py3-none-any.whl
```

2.2 智能体编程

2.2.1 测试驱动的智能体修复流程

先编写一个失败测试明确期望行为，再让智能体以该测试为验收标准修改代码，可形成可验证的修复闭环。

```
# 1. 先写失败测试并运行，确认失败
pytest test_cli.py
# 2. 让智能体修改实现，以"运行 pytest 通过"为约束
# 3. 再次运行测试，确认修复有效
pytest test_cli.py
```

2.2.2 用 git diff 审查智能体的改动

智能体可能引入与目标无关的改动，提交前应使用 git diff 检查，撤销无关修改。

```
git diff          # 查看工作区与暂存区的差异
git diff --stat   # 仅查看改动文件与行数
git checkout -- <file> # 撤销某个文件的无关修改
```

2.3 不止于代码

2.3.1 编写规范的 Git 提交信息

采用 Conventional Commits 约定, 标题用祈使语气并标注类型 (fix/feat/docs 等), 正文说明问题与方案, 便于历史回溯。

```
git commit -m "fix: 拦截纯空白姓名的无效输入" \
            -m "当前 --name 未校验空白字符, 导致错误问候且退出码为 0。"
```

2.3.2 编写可复现的 Issue 报告

一份可执行的 Issue 应包含环境、复现命令、期望结果与实际结果, 未知信息明确写“待确认”, 便于他人快速定位。

```
### Issue: 空姓名仍输出问候语
- 环境: Windows 10, Python 3.10+
- 复现命令: sdt-greet --name "  "
- 预期结果: 报错并以非零状态码退出。
- 实际结果: 输出问候语并以 0 退出。
```

2.4 Python 与 PyTorch

2.4.1 PyTorch 张量的创建与运算

张量是 PyTorch 的基本数据结构, 支持与 NumPy 类似的创建方式与广播运算, 入门可参考课程资源中的 PyTorch 官方教程。

```
import torch
a = torch.linspace(-1, 1, 5)  # 等间隔张量
x = a.reshape(-1, 1)         # 改变形状
y = 3 * x - 1                 # 广播运算
print(y)
```

2.4.2 自动求导与梯度清零

PyTorch 通过 backward 自动计算梯度，但梯度默认累加，因此每次迭代前需调用 zero_grad 清空。

```
import torch
x = torch.tensor([2.0], requires_grad=True)
y = x * x                      #  $y = x^2$ 
y.backward()                   # 自动求导  $dy/dx = 2x$ 
print(x.grad)                  # tensor([4.])
opt.zero_grad()                # 清空累积的梯度
```

2.4.3 用 nn.Linear 搭建最小训练循环

nn.Linear 表示线性层，配合损失函数与优化器即可构成最小训练循环，三步完成一次参数更新。

```
import torch
from torch import nn

model = nn.Linear(1, 1)
loss_fn = nn.MSELoss()
opt = torch.optim.SGD(model.parameters(), lr=0.1)
```

```
pred = model(x)
loss = loss_fn(pred, y)
opt.zero_grad()    # 清空梯度
loss.backward()    # 反向传播
opt.step()         # 更新参数
```

3 感悟

通过第三周的实验练习，形成以下认识和体会：

- 打包与分发的重要性：借助 `pyproject.toml` 与 `python -m build`，可以把一个最小命令行包标准化地构建为 `wheel`，并在全新虚拟环境中仅凭该 `wheel` 完成安装与运行，保证了交付的可复现性。
- 测试驱动的智能体协作：先写失败测试、再让智能体以测试通过为目标修改实现，能够形成可验证的闭环；但智能体的改动仍需人工用 `git diff` 审查，防止引入无关修改。
- 协作材料的可执行性：一份好的 `Issue`、提交信息与评审意见应当给出环境、复现步骤、期望与实际结果，并标注风险级别，才能让协作者无需反复追问即可行动。
- PyTorch 训练循环的闭环：一次完整的参数更新包含 `zero_grad`、`backward`、`step` 三步；梯度若不清空会逐次累加，理解这一点是正确使用自动求导的前提。

从把代码打包分发，到与智能体协作，再到撰写高质量协作材料和使用深度学习框架，第三周的内容从“代码如何交付”延伸到了“如何与他人、与工具高效协作”，体现了工程实践不止于写出能运行的代码，更在于让代码与协作过程都可复现、可验证、可维护。

Github 提交

此处上传 Github 提交记录。(报告于完成后上传)

